

NEXHIRE

Technical Architecture, Data and API Specification

*Solution Architecture, Database, APIs, Security, Cloud and Implementation
Standards*

Volume 3 of 4 | Version 3.0 | Team ALPHA | 5 August 2026

Enterprise Second-Brain Edition - Controlled Source of Truth

Document Control and Version History

Field	Value
Product	NexHire Smart Recruitment Matching Platform
Document Set	Enterprise Documentation and Second Brain v3.0
Owner	Kogul / Team ALPHA
Classification	Project Internal / Academic Submission / Client-Ready
Canonical Product Baseline	Second Brain Final Pack v2.0
Implementation Overlay	Master Documentation and Team Handover Pack
Review Rule	Changes require an approved decision record and traceability update

Version	Date	Status	Summary
1.0	4 Aug 2026	Historical	Original source documentation baseline
2.0	5 Aug 2026	Validated Source	Second Brain canonical pack and validated prototype
3.0	5 Aug 2026	Current Controlled Edition	Unified enterprise documentation with conflicts resolved through phase boundaries

Authoritative interpretation rule

Core Release v1.0 follows the trust-first, explainable matching, duplicate-safe application and consent-gated contact scope of the validated Second Brain pack. Interview, Offer, Talent Pool and Complaint capabilities from the implementation overlay are retained as Phase 2 extensions, not silently mixed into Core Release v1.0.

Table of Contents

25	Architecture Overview
26	Technology Stack and Decisions
27	Logical and Module Architecture
28	Database Design and ER Model
29	API Architecture and Endpoint Catalogue
30	Authentication and Authorization
31	Security Architecture
32	Performance, Scalability and Reliability
33	Error Handling, Logging and Audit
34	Cloud and Deployment Architecture
35	Backup and Disaster Recovery
36	Coding Standards and Folder Structure

37 Third-Party Integrations

38 Appendices

25. Architecture Overview

NexHire uses a modular-monolith architecture aligned with Clean Architecture principles. The API is stateless, domain policies are explicit, relational data is authoritative, private files are separated into object storage, and background processing handles durable delivery and lifecycle expiry. This design is appropriate for the initial team and university delivery while preserving a credible enterprise deployment target.

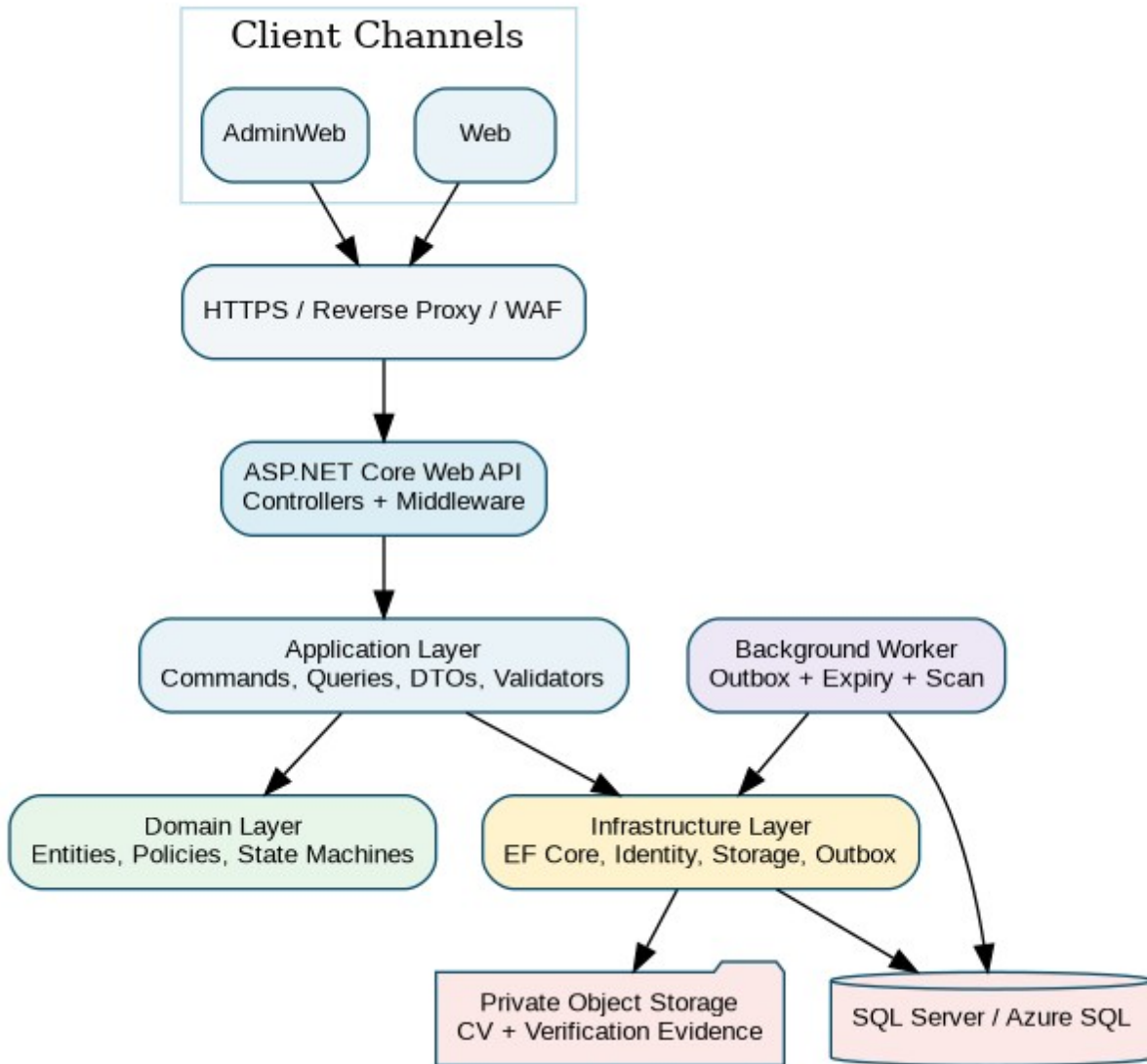


Figure 4 - Logical architecture and dependency direction

25.1 Dependency Rule

The Domain layer depends on no infrastructure framework. The Application layer depends on Domain and abstractions. Infrastructure implements persistence, identity, storage and external service abstractions. API composes dependencies and exposes transport contracts. No controller directly implements complex business rules.

26. Technology Stack and Architecture Decisions

Area	Approved Direction	Reasoning
Frontend	Responsive web client using standards-based HTML/CSS/JavaScript or an approved framework	Supports prototype continuity and phased modernization.
Backend	ASP.NET Core Web API	Strong typed contracts, middleware, Identity and enterprise tooling.
Runtime Version	Versioned LTS/current baseline selected by repository release plan	Avoids making a product rule depend on a transient SDK version.
Persistence	EF Core with SQL Server/Azure SQL	Relational constraints, transactions, concurrency and reporting.
Files	Private object storage abstraction	Separates binary lifecycle from relational metadata and enables cloud migration.
Authentication	ASP.NET Core Identity + short-lived JWT + rotating refresh token	Secure role sessions and stateless API scale.
Architecture	Modular Monolith + Clean Architecture	Low operational complexity with strong boundaries and extraction path.
Async Effects	Transactional Outbox + Worker	Prevents business commit/notification inconsistency.
Observability	Structured logs, metrics and distributed traces	Operational diagnosis without sensitive content.

26.1 Key Architecture Decision Records

ADR	Decision	Reason
ADR-001	Use modular monolith	Best balance of team size, transaction integrity and future modular extraction.
ADR-002	Use Clean Architecture boundaries	Protects domain rules and enables testing.
ADR-003	Use deterministic RM-1.0	Explainable, replayable and appropriate for initial evidence quality.
ADR-004	Use private object storage	CV/evidence requires different access and retention controls from SQL.
ADR-005	Use outbox pattern	Business state and durable notification intent commit together.
ADR-006	Use optimistic concurrency	Prevents silent overwrites in verification, vacancy, application and consent decisions.
ADR-007	Use consent-gated contact	Protects candidate agency and reduces PII leakage.
ADR-008	Phase-label extended recruitment modules	Prevents contradictory MVP scope.

27. Logical and Module Architecture

Module	Name	Responsibility
M01	Identity and Access	Registration, login, refresh, logout, role and account-state enforcement.
M02	Candidate Profile	Structured candidate evidence and readiness.
M03	Skill Passport	Canonical skills, aliases and candidate evidence.
M04	Protected CV and CV Studio	Private upload, scan, controlled streaming and deterministic template rendering.
M05	Company Trust	Company profile, evidence and verification lifecycle.
M06	Vacancy Lifecycle	Draft, moderation, activation, closure and

Module	Name	Responsibility
		expiry.
M07	Search and Discovery	Trusted public vacancy search and candidate projections.
M08	Matching Engine	RM-1.0 deterministic scoring, gaps, receipts and replay.
M09	Applications	Idempotent duplicate-safe apply, history and withdrawal.
M10	Employer Applicant Review	Stable ranking, evidence review and protected CV access.
M11	Application Status	Allowed transition matrix, history and notifications.
M12	Consent Contact	One pending request, candidate decision, conditional detail release.
M13	Notifications	Owner-only in-app inbox and durable outbox delivery.
M14	Privacy and Deletion	Preferences and controlled deletion requests.
M15	Reporting and Audit	Truth-based summaries, exports and append-only audit.
M16	Administration and Governance	Verification, users, moderation, skill governance and locked settings.

27.1 Request Processing Pipeline

1. HTTPS request reaches reverse proxy/WAF.
2. Correlation, security headers and rate-limit middleware run.
3. Authentication validates access token and account state.
4. Authorization evaluates role and resource ownership.
5. Model and application validation execute.
6. Application handler invokes domain policy and repository abstraction.
7. EF Core transaction commits aggregate, history, idempotency and outbox.
8. Response DTO or Problem Details is returned.
9. Worker delivers asynchronous effects and lifecycle jobs.

28. Database Design and ER Model

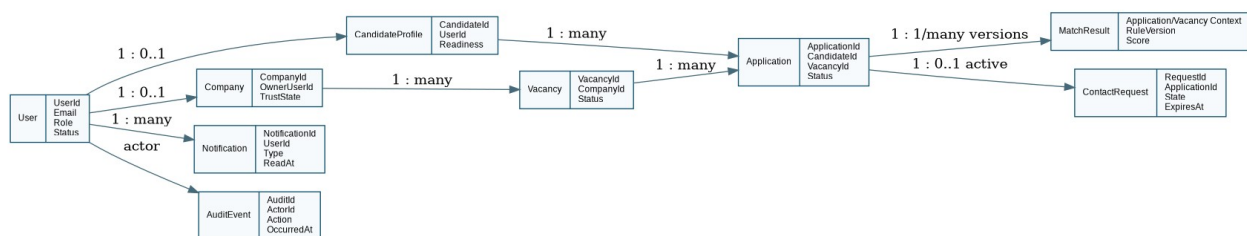


Figure 5 - Core entity relationship overview

28.1 Database Design Principles

- Use UUID/GUID or approved stable identifiers consistently.
- Store timestamps in UTC.
- Use RowVersion/concurrency token on mutable decision aggregates.
- Enforce ownership and uniqueness through keys and constraints, not only code.
- Use normalized canonical skills and aliases.

- Separate append-only history/audit from current aggregate state.
- Store binary metadata in SQL and binary content in private object storage.
- Use indexes based on API query patterns and bounded pagination.

Table / Aggregate	Purpose	Key Constraints
AspNetUsers / Roles	Identity, role and account state	Unique normalized email; account-state policy
RefreshTokens	Rotating refresh sessions	Hashed token, expiry, revocation, reuse detection
CandidateProfiles	Structured candidate evidence	One per Job Seeker user
PrivacyPreferences	Visibility and contact/deletion preferences	One per candidate; concurrency token
Skills / SkillAliases	Canonical skill dictionary	Unique normalized names and aliases
CandidateSkills	Candidate evidence against canonical skill	Unique CandidateId + SkillId
ResumeFiles	Private CV metadata and scan state	One active current policy; storage key isolated
CvBuilderDocuments	Structured CV Studio JSON	Schema version and concurrency token
Companies	Employer organization and trust state	Owned by Employer user; verification-derived trust
VerificationReviews	Evidence and admin decisions	Append decision history and reason
Vacancies	Vacancy aggregate and lifecycle	Company FK, status, dates, concurrency
VacancySkills	Required/preferred canonical skill links	Unique VacancyId + SkillId + kind
MatchRuleVersions	Immutable scoring configuration	Versioned, approved and activation metadata
MatchResults	Deterministic score receipt	Inputs/rule version/timestamp/total
MatchResultSkills	Matched/gap detail	Trace each required skill
Applications	Candidate-vacancy transaction	Unique CandidateId + VacancyId
ApplicationStatusHistory	Ordered status history	Append-only sequence/timestamp/actor
ContactRequests	Job-specific consent request	One active/request policy, expiry, decision
Notifications	Owner inbox	User FK, type, read timestamp
OutboxMessages	Durable async intent	Processing state and retry count
IdempotencyRecords	Command replay protection	Key + actor + route scope
VacancyReports	Candidate vacancy reports	Reporter/vacancy uniqueness policy
ModerationHistory	Admin vacancy decisions	Append-only reason/action
DeletionRequests	Controlled privacy request	Candidate and lifecycle state
AuditEvents	Immutable accountability	Actor/action/resource/time/correlation

28.2 Relationship Explanation

- A User may own one CandidateProfile or one Company context based on role.
- A Company owns many Vacancies.
- A Candidate and Vacancy are connected through Applications with a lifetime unique key in Core Release.
- A MatchResult records the rule-versioned explanation used for a candidate-vacancy/application context.
- ApplicationStatusHistory is append-only and ordered.
- A ContactRequest references an eligible application and controls conditional contact projection.
- Notifications and AuditEvents reference users/resources but retain immutable event meaning.

29. API Architecture and Endpoint Catalogue

29.1 API Standards

- Versioned base path /api/v1.
- JSON uses consistent naming and UTC ISO-8601 timestamps.
- Private file endpoints use streaming and content-disposition controls.
- Pagination is bounded and stable.

- If-Match protects mutable resources.
- Idempotency-Key protects repeatable commands.
- Problem Details provides stable error codes.
- OpenAPI is generated, reviewed and contract-tested.

Method	Path	Authorization	Purpose	Use Case	Outcomes
POST	/api/v1/auth/register	Anonymous	Register JobSeeker or Employer; acknowledge notice	UC01	201/400/409/429
POST	/api/v1/auth/login	Anonymous	Authenticate active account and issue access/refresh session	UC02	200/400/401/429
POST	/api/v1/auth/refresh	Refresh cookie	Rotate hashed refresh token and issue new access token	UC02	200/401/409/429
POST	/api/v1/auth/logout	Authenticated/cookie	Revoke refresh session; idempotent	UC02	204/401
GET	/api/v1/auth/me	Authenticated	Return minimal identity, role and workspace state	UC02	200/401
GET	/api/v1/public/jobs	Anonymous	Search only trusted public vacancies	UC07	200/400/429
GET	/api/v1/public/jobs/{vacancyId}	Anonymous	Trusted vacancy public detail	UC07	200/404/429
GET	/api/v1/skills	Authenticated	Canonical skill/alias lookup with paging	UC04, UC15, UC23	200/400/401/429
GET	/api/v1/candidate/profile	JobSeeker owner	Read structured profile and readiness	UC03	200/401/403
PUT	/api/v1/candidate/profile	JobSeeker owner	Replace validated profile using If-Match	UC03	200/400/401/409/422
GET	/api/v1/candidate/skills	JobSeeker owner	Read confirmed skill passport	UC04	200/401/403
POST	/api/v1/candidate/skills	JobSeeker owner	Add canonical skill and evidence level	UC04	201/400/409/422
DELETE	/api/v1/candidate/skills/{skillId}	JobSeeker owner	Remove confirmed skill evidence	UC04	204/404/409
GET	/api/v1/candidate/resume	JobSeeker owner	Read current CV metadata only	UC05	200/404
POST	/api/v1/candidate/resume	JobSeeker owner	Upload multipart CV to quarantine; max 5 MB	UC05	201/400/413/415/422/429
GET	/api/v1/candidate/resume/content	JobSeeker owner	Authorized private stream of own active CV	UC05	200/404/423
DELETE	/api/v1/candidate/resume	JobSeeker owner	Deactivate current CV and remove eligible object	UC05	204/404/409
GET	/api/v1/candidate/cv-document	JobSeeker owner	Read candidate-authored CV Studio document	UC06	200/404
PUT	/api/v1/candidate/cv-document	JobSeeker owner	Save schema-validated CV Studio draft	UC06	200/400/409/422
POST	/api/v1/candidate/cv-document/render	JobSeeker owner	Render candidate text into approved PDF template	UC06	201/400/422/429
GET	/api/v1/candidate/jobs	JobSeeker	Trusted vacancies plus deterministic match projection	UC07, UC08	200/400/401/429
GET	/api/v1/candidate/jobs/{vacancyId}/match	JobSeeker	RM-1.0 components, gaps and replay receipt	UC08	200/404/409/422
POST	/api/v1/candidate/applications	JobSeeker	Duplicate-safe apply; Idempotency-Key required	UC09	201/400/404/409/422/429
GET	/api/v1/candidate/applications	JobSeeker owner	Paged application tracker	UC10	200/400
GET	/api/v1/candidate/applications/{applicationId}	JobSeeker owner	Application detail and ordered history	UC10	200/404
POST	/api/v1/candidate/applications/{applicationId}/withdraw	JobSeeker owner	Allowed-state withdrawal command	UC10	200/404/409/412
GET	/api/v1/candidate/contact-requests	JobSeeker owner	Paged job-specific consent requests	UC11	200/400
POST	/api/v1/candidate/	JobSeeker owner	Accept or decline once	UC11	200/404/409/412/422

Method	Path	Authorization	Purpose	Use Case	Outcomes
	contact-requests/{requestId}/decision		using If-Match		
GET	/api/v1/notifications	Authenticated owner	Paged in-app inbox and unread count	UC12	200/400
POST	/api/v1/notifications/{notificationId}/read	Authenticated owner	Mark one notification read; idempotent	UC12	204/404
POST	/api/v1/notifications/read-all	Authenticated owner	Mark current inbox read; bounded command	UC12	204/429
GET	/api/v1/candidate/privacy	JobSeeker owner	Read privacy notice, visibility and request preferences	UC13	200
PUT	/api/v1/candidate/privacy	JobSeeker owner	Update visibility/preferences using If-Match	UC13	200/400/409/412
POST	/api/v1/candidate/deletion-requests	JobSeeker owner	Submit controlled deletion request	UC13	202/400/409
GET	/api/v1/candidate/deletion-requests/latest	JobSeeker owner	Read request state without internal retention detail	UC13	200/404
GET	/api/v1/employer/company	Employer owner	Read owned company and trust state	UC14	200/404
PUT	/api/v1/employer/company	Employer owner	Update owned company using If-Match	UC14	200/400/409/412
POST	/api/v1/employer/company/verification	Employer owner	Submit evidence for manual verification	UC14	202/400/409/413/415
GET	/api/v1/employer/company/verification	Employer owner	Read current/history-safe verification status	UC14	200/404
GET	/api/v1/employer/vacancies	Employer owner	Paged owned vacancy workspace	UC15	200/400
POST	/api/v1/employer/vacancies	Employer owner	Create Draft vacancy	UC15	201/400/409/422
GET	/api/v1/employer/vacancies/{vacancyId}	Employer owner	Read owned vacancy detail	UC15	200/404
PUT	/api/v1/employer/vacancies/{vacancyId}	Employer owner	Update permitted vacancy fields with If-Match	UC15	200/400/404/409/412/422
POST	/api/v1/employer/vacancies/{vacancyId}/activate	Employer owner	Publish only when full trust predicate passes	UC15	200/404/409/412/422
POST	/api/v1/employer/vacancies/{vacancyId}/close	Employer owner	Close owned Active vacancy; no v1 reopen	UC15	200/404/409/412
GET	/api/v1/employer/vacancies/{vacancyId}/applicants	Employer vacancy owner	Stable RM-1.0 ranked applicant queue	UC16	200/400/404
GET	/api/v1/employer/applications/{applicationId}	Employer vacancy owner	Applicant evidence, symmetric explanation, masked contact	UC16	200/404
GET	/api/v1/employer/applications/{applicationId}/resume	Employer vacancy owner	Authorized private CV stream and audit	UC16	200/404/409/423
POST	/api/v1/employer/applications/{applicationId}/status	Employer vacancy owner	Allowed transition + history + notification	UC17	200/404/409/412/422
POST	/api/v1/employer/applications/{applicationId}/contact-request	Employer vacancy owner	Create one Pending 7-day request; Idempotency-Key	UC18	201/404/409/422/429
GET	/api/v1/employer/contact-requests	Employer owner	Paged request states	UC18	200/400
GET	/api/v1/employer/contact-requests/{requestId}/accepted-details	Employer context owner	Release phone/email only after Accepted and valid context	UC18	200/404/409
GET	/api/v1/employer/reports/summary	Employer owner	Database-truth basic recruitment summary	UC19	200/400/429
GET	/api/v1/employer/reports/applications.csv	Employer owner	Bounded formula-safe CSV export	UC19	200/400/413/429
GET	/api/v1/admin/verifications	Administrator	Paged Pending company verification queue	UC20	200/400
GET	/api/v1/admin/verifications/{reviewId}	Administrator	Evidence and history for decision	UC20	200/404

Method	Path	Authorization	Purpose	Use Case	Outcomes
POST	/api/v1/admin/verifications/{reviewId}/decision	Administrator	Verify or reject with reason using If-Match	UC20	200/400/404/409/412
GET	/api/v1/admin/dashboard	Administrator	Truth counts: users, vacancies, applications	UC21	200/429
GET	/api/v1/admin/users	Administrator	Bounded account query	UC21	200/400
POST	/api/v1/admin/users/{userId}/suspend	Administrator	Suspend account; revoke sessions; audit	UC21	200/404/409/412
POST	/api/v1/admin/users/{userId}/reactivate	Administrator	Reactivate eligible account; audit	UC21	200/404/409/412
POST	/api/v1/public/jobs/{vacancyId}/reports	Authenticated JobSeeker	Report a trusted public vacancy	UC22	202/400/404/409/429
GET	/api/v1/admin/vacancy-reports	Administrator	Moderation/report queue	UC22	200/400
POST	/api/v1/admin/vacancies/{vacancyId}/moderation	Administrator	Approve/reject/close with reason	UC22	200/400/404/409/412
POST	/api/v1/admin/vacancy-reports/{reportId}/resolve	Administrator	Resolve one report and audit outcome	UC22	200/400/404/409
POST	/api/v1/admin/skills	Administrator	Create unique canonical skill	UC23	201/400/409
PUT	/api/v1/admin/skills/{skillId}	Administrator	Rename/deactivate safely with If-Match	UC23	200/400/404/409/412
POST	/api/v1/admin/skills/{skillId}/aliases	Administrator	Add unique normalized alias	UC23	201/400/404/409
DELETE	/api/v1/admin/skills/{skillId}/aliases/{aliasId}	Administrator	Remove alias without deleting candidate evidence	UC23	204/404/409
GET	/api/v1/admin/audit-events	Administrator audit policy	Bounded append-only audit query	UC24	200/400/403/429
GET	/api/v1/admin/settings	Administrator	Read locked platform/scoring settings	UC24	200/403

29.2 Current Implementation Compatibility Mapping

Canonical Contract	Current Overlay Example	Rule
GET /api/v1/candidate/applications	GET /api/Applications/mine	Treat current route as implementation adapter until v1 contract migration.
GET /api/v1/candidate/profile	GET /api/JobSeekers/me	DTO and ownership semantics must remain canonical.
GET /api/v1/employer/company	GET /api/Companies/me	Map exact repository names without changing business meaning.
GET /api/v1/admin/dashboard	GET /api/Dashboard/admin	Dashboard metric definitions must reconcile.

30. Authentication and Authorization

30.1 Authentication Flow

10. User submits credentials over HTTPS.
11. Identity verifies active account and password hash.
12. API issues short-lived access token and rotating refresh token/cookie.
13. Refresh rotates token and detects replay.
14. Logout revokes session idempotently.
15. Account suspension revokes active sessions.

30.2 Authorization Model

Control	Requirement
Role Policy	JobSeeker, Employer and Administrator policies gate

Control	Requirement
	workspace-level operations.
Resource Ownership	Every identifier is checked against candidate/company/vacancy/application context.
State Policy	Action is allowed only in current aggregate state.
Private File Policy	Caller, business context and scan state are checked before streaming.
Contact Policy	Accepted unexpired request and valid application context are required.
Admin Separation	Admin trust operations do not imply score override permission.

31. Security Architecture

Threat / Control Area	Required Controls
Credential attacks	Identity lockout/rate limiting, secure password policy, no credential logging.
Token theft/replay	Short access lifetime, rotating hashed refresh tokens, revocation and reuse detection.
Broken object authorization	Resource-based policies and existence-hiding 404 where appropriate.
File upload attack	Extension/MIME/signature checks, size limit, quarantine, malware scan, private storage.
PII leakage	Mask by default, consent-gated projection, output DTO minimization, audit.
Injection	Parameterized EF queries, output encoding, formula-safe CSV.
CSRF/CORS	Appropriate cookie protection and restricted production origins.
Race conditions	Unique constraints, transactions, optimistic concurrency and idempotency.
Abusive automation	Rate limits, bounded exports and operational alerting.
Audit tampering	Append-only audit permissions and retention controls.

31.1 Secure Development Requirements

- Threat-model every new module.
- Dependency and secret scanning in CI.
- No production secrets in source or client code.
- Security tests cover role, ownership, state and private file access.
- Logs redact tokens, passwords, CV content, contact data and verification evidence.

32. Performance, Scalability and Reliability

32.1 Performance Considerations

- Use projection queries and AsNoTracking for reads.
- Index public trust predicate, vacancy search filters, application uniqueness and status queues.
- Avoid N+1 navigation loading in dashboards/ranking.
- Cache only safe reference data such as canonical skills; invalidate by version.
- Stream files; do not load full binaries into application memory.
- Bound exports, list sizes and admin audit queries.

32.2 Scalability

The stateless API can scale horizontally behind a load balancer. SQL remains the transactional source of truth. Private object storage scales independently. Worker instances use leasing/locking to prevent

duplicate lifecycle processing. Module boundaries allow later extraction only when demonstrated load or organizational needs justify distributed services.

32.3 Reliability

- Transactional outbox for asynchronous effects.
- Retry with exponential backoff and poison-message visibility.
- Idempotent notification/read/logout commands.
- Health probes distinguish liveness and dependency readiness.
- Timeouts and cancellation tokens on I/O operations.

33. Error Handling, Logging and Audit Trail

33.1 Exception Handling

A global exception middleware maps known domain/application errors to stable Problem Details. Unexpected exceptions are logged with correlation data and return a generic safe response. Missing optional profile data returns a defined zero/empty state where the contract permits it, not an unhandled exception.

33.2 Logging

Log Type	Content
Request	Method, route template, status, duration, correlation, actor id where safe.
Business Event	Application submitted, status changed, consent decided, verification/moderation result.
Security	Authentication failure category, authorization denial aggregate, token replay signal.
Operational	Outbox retry, scan failure, worker lease, dependency health.
Prohibited	Passwords, access/refresh tokens, CV bodies, raw verification files, unmasked contact details.

33.3 Audit Trail

Audit is business accountability, not diagnostic logging. Audit events are append-only and record actor, action, resource type/id, outcome, reason where applicable, timestamp, rule/version context and correlation identifier.

34. Cloud and Deployment Architecture

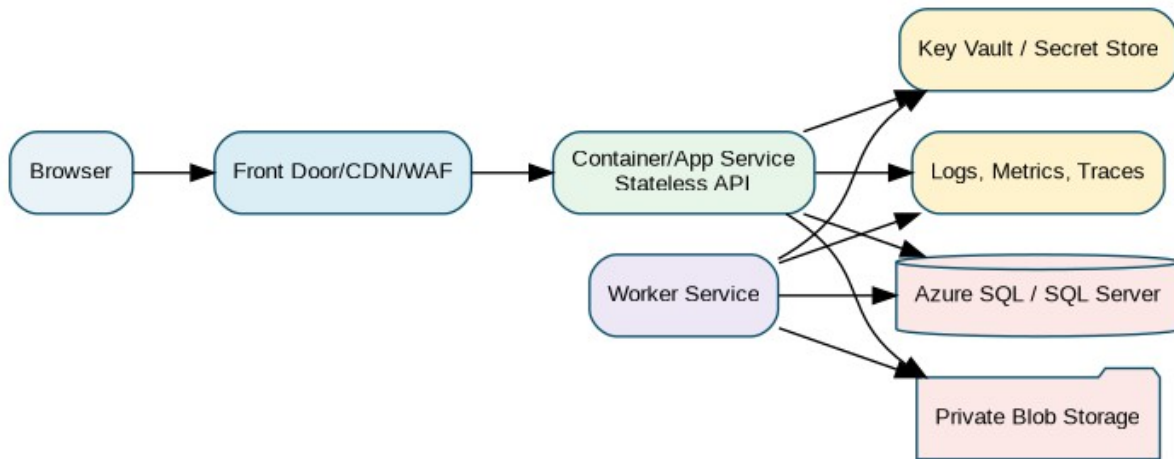


Figure 6 - Production target deployment architecture

34.1 Environment Model

Environment	Purpose	Controls
Local	Developer iteration	Local SQL/storage emulator, development secrets, seeded test data.
Test/CI	Automated verification	Ephemeral database, contract/security tests, no production data.
Staging	Production-like acceptance	Restricted access, production topology, synthetic/sanitized data.
Production	Live service	WAF, private data-plane access, managed secrets, backup, monitoring and incident process.

34.2 MVP Deployment Option

For the university delivery, the API, worker and SQL Server may run in a simplified controlled topology, provided the same security boundaries, private file abstraction, migration process, logging and backup expectations are demonstrated. The simplified deployment is not presented as equivalent proof of the production target.

35. Backup Strategy and Disaster Recovery

Asset	Backup / Recovery Requirement
SQL Database	Automated full/differential/log or managed PITR, encrypted backup, periodic restore test.
Private Objects	Versioning/soft delete where supported, retention alignment and restore exercise.
Configuration	Version-controlled non-secret config and recoverable secret references.
OpenAPI and Documentation	Versioned release artifacts stored with source control/release package.
RPO/RTO	Approved before production; MVP recommendation documented and tested.

35.1 Recovery Procedure

16. Declare incident and stop harmful writes if required.
17. Select recovery point and validate legal/retention constraints.
18. Restore database and object references in isolated environment.
19. Run integrity checks for application, history, match receipt, contact and audit data.
20. Reconcile outbox/worker state and resume safely.
21. Record evidence and corrective actions.

36. Coding Standards and Folder Structure

36.1 Backend Folder Structure

```
NexHire.sln
src/
  NexHire.API/      Controllers, middleware, filters, composition
  NexHire.Application/ DTOs, commands, queries, interfaces, validators
  NexHire.Domain/   Entities, enums, policies, state machines
  NexHire.Infrastructure/EF Core, repositories, Identity, storage, outbox
tests/
  NexHire.UnitTests/
  NexHire.IntegrationTests/
frontend/
  auth/ admin/ company/ jobseeker/ assets/
docs/ contracts/ deployment/
```

36.2 Coding Rules

- PascalCase public C# symbols; camelCase locals; descriptive names.
- Async I/O uses Async suffix and CancellationToken.
- Controllers are thin and return DTOs, not EF entities.
- Domain rules are not duplicated in JavaScript.
- Enums/state transitions are centralized and versioned.
- Queries avoid string ToString comparisons in database predicates.
- Frontend API wrapper centralizes token, timeout, parsing and error formatting.
- All source-derived HTML output is escaped.
- Shared files require lead approval to reduce team mismatch.

37. Third-Party Integrations

Core Release requires no mandatory external recruitment provider. Infrastructure abstractions are designed for object storage, malware scanning, email/SMS and observability providers, but activation requires a documented data-flow, security review, failure strategy, cost model and contract.

Integration	Core Status	Architecture Requirement
Object Storage	Required abstraction	Private access, generated key, encryption, lifecycle and audit.
Malware Scanner	Required production capability	Quarantine-to-active workflow and failure/timeout policy.
Email/SMS	Optional / Phase decision	Outbox adapter, opt-in/notification preference, retry and no sensitive content.
Calendar	Phase 2 Interview extension	Time zone, cancellation and token/privacy review.
Identity Federation	Future	OIDC/SAML threat model and account

Integration	Core Status	Architecture Requirement
		linking.
Analytics	Internal safe metrics only	No protected-characteristic ranking analytics.

38. Appendices

Appendix C - API Review Checklist

- Route versioned and named consistently.
- Role/resource/state authorization documented.
- Request/response schemas and examples available in OpenAPI.
- Validation and stable Problem Details codes defined.
- Idempotency and concurrency requirements identified.
- Database constraints support business invariant.
- Sensitive fields minimized and masked.
- Unit, integration, security and contract tests linked.

38.3 v3.1 Amendment - Verification Data, API and Security Contract

Area	v3.1 Approved Rule
Data contract	CompanyVerificationSubmission, CompanyVerificationDocument, CompanyVerificationDecision and CompanyVerificationHistory are separate concepts. Documents are private objects; the relational database stores metadata, status, hash and review attributes.
API contract	Employer endpoints support profile, evidence metadata, upload initiation/completion, submission, status and resubmission. Admin endpoints support queue, detail, document stream authorization and decision commands.
Security	Files require extension, MIME, size, signature and malware validation. Admin document access is resource-authorized and audited. Candidate/public APIs never expose verification documents or representative identity data.
Concurrency	Submission and decision commands use RowVersion/If-Match. Duplicate submissions are idempotently rejected while a review is active.
Audit	Every upload, replacement, submission, view, request-more-information, rejection, approval, suspension and re-verification event is auditable.

This amendment is authoritative for Company Verification. Where an earlier summary is less detailed, this amendment and Volume 05 provide the complete implementation and UX contract.